

# Selectores Fantasma y la Guerra de Especificidad CSS en SASS

**Estantería:** Desarrollo y Arquitectura

**Subtema:** Estilos

**mercedev.es** — 2026-05-26 | Epic 6 - Fase 1

# Selectores Fantasma y la Guerra de Especificidad CSS en SASS

## El Desafío

---

Al inyectar estilos personalizados para domar la vista individual de un producto en WooCommerce, el navegador ignoraba sistemáticamente las reglas CSS con `!important`. La imagen del producto permanecía invisible (`opacity: 0` inyectado en línea por el CMS) a pesar de que el código SASS parecía sintácticamente perfecto.

El análisis forense reveló dos problemas que actuaban en conjunto: una especificidad débil que perdía frente a los estilos residuales del CMS, y un bloque entero de CSS convertido en "Código Muerto" inalcanzable por el DOM.

## La Maniobra

---

Se aplicaron dos correcciones quirúrgicas en la arquitectura SASS (`_woocommerce.scss`):

- Corrección de anidamiento (Nesting):** Se detectó que el selector `.single-product &` compilaba con un espacio (`.single-product .woocommerce`), obligando al navegador a buscar un contenedor dentro de otro. Como WordPress inyecta ambas clases en la misma etiqueta `<body>`, el selector fallaba. Se invirtió a `&.single-product`, lo que compila como `.woocommerce.single-product` (sin espacio), apuntando directamente al elemento raíz.
- Aumento de especificidad:** Los bloques internos (`div.images`, `div.summary`) estaban flotando en la raíz del componente. Se movieron e anidaron estrictamente dentro de `div.product`, generando un selector combinado ultra-específico

( `.single-product .woocommerce div.product div.images` ) capaz de destruir cualquier estilo residual.

## El Aprendizaje / Deuda Técnica

---

En arquitecturas CSS BEM potenciadas por SASS, el símbolo *ampersand* ( `&` ) es un arma de doble filo. No comprender cómo el compilador traduce la anidación genera **Selectores Fantasma**: código CSS que se descarga y procesa, pero que el navegador jamás aplica porque el DOM no coincide con la jerarquía esperada.

Además, cuando se combate contra frameworks pesados o CMS que abusan de estilos por defecto, la solución rara vez es añadir más `!important`. La verdadera victoria se logra aumentando el peso matemático del selector en la cascada CSS mediante un anidamiento estricto y contextual.